`-e ssh` instead.

It has been pointed out that rsync operates far more efficiently in server mode than it does over NFS, so if the connection between your source and backup server becomes a bottleneck, you should consider configuring the backup machine as an rsync server instead of using NFS. On the downside, this approach is slightly less transparent to users than NFS – snapshots would not appear to be mounted as a system directory, unless NFS is used in that direction, which is certainly another option (We haven't tried it yet though). Thanks to Martin Pool, a lead developer of rsync, for making me aware of this issue.

Here's another example of the utility of this approach – one that we use. If you have a bunch of Windows desktops in a lab or office, an easy way to keep them all backed up is to share the relevant files, read-only, and mount them all from a dedicated backup server using SAMBA. The backup job can treat the SAMBA-mounted shares just like regular local directories.

Run it all with cron

Cron is an administrator's dream, it is simple, fast and accurate task scheduler.

To schedule any task to run once at a later time or recurrently as in the case of our backup, we can schedule it via cron. The first step towards that would be to edit the crontab file. The crontab file is the configuration file for cron jobs, we will edit the crontab of root by opening the file

```
$ vi /etc/crontab
```

The above is the system wide cron file and you would need root permissions to edit it.

To list the cronjobs for current user, issue the following command

```
$ crontab -l
```

A little introduction to crontab format should be of help here.

A cron job consists out of six fields:

```
<minute> <hour> <day of month> <month> <day of week>
<command>
```

```
field           allowed values
-----           --------------
minute          0-59
hour            0-23
day of month    1-31
month           1-12 (or names, see below)
day of week     0-7 (0 or 7 is Sun, or
use names)
```